

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

Кафедра
інформатики та програмної інженерії
(повна назва кафедри, циклової комісії)

КУРСОВА РОБОТА

з Основ програмування

(назва дисципліни)

на тему: «Арифметика довгих чисел»

Студентки І курсу, групи ІП-54
Шугаєвої Поліни Олексіївни

Спеціальності 121 «Інженерія програмного
забезпечення»

ст. в. Вітковська І.І.
(посада, вчене звання, науковий
ступінь, прізвище та ініціали)

Кількість балів:

Національна оцінка

Члени комісії

ст. в. Вітковська І.І.

(підпис)

(посада, вчене звання, науковий ступінь,
прізвище та ініціали)

асистент Вовк Є.А.

(підпис)

(посада, вчене звання, науковий ступінь,
прізвище та ініціали)

Київ – 2026 рік

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

(назва вищого навчального закладу)

Кафедра інформатики та програмної інженерії

Дисципліна Основи програмування. Курсова робота

Напрямок «ІПЗ»

Курс 1 Група ІІІ-54

Семестр 2

ЗАВДАННЯ

на курсову роботу студента

Шугаєвої Поліни Олексіївни

(прізвище, ім'я, по батькові)

1. Тема роботи «Арифметика довгих чисел»

2. Строк здачі студентом закінченої роботи _____

3. Вихідні дані до роботи Додаток А Технічне завдання

4. Зміст пояснювальної записки (перелік питань, які підлягають розробці)

Постановка задачі, теоретичні відомості, опис алгоритмів, опис програмного забезпечення,

тестування програмного забезпечення, інструкція користувача, аналіз результатів.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6. Дата видачі завдання «13» лютого 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсової роботи	Термін виконання етапів роботи	Підписи керівника, студента
1.	Отримання теми курсової роботи	13.02.2026	
2.	Підготовка ТЗ	15.02.2026	
3.	Пошук та вивчення літератури з питань курсової роботи	01.03.2026	
4.	Розробка сценарію роботи програми	14.03.2026	
6.	Узгодження сценарію роботи програми з керівником	20.03.2026	
5.	Розробка (вибір) алгоритму розв'язання задач	28.03.2026	
6.	Узгодження алгоритму з керівником	02.04.2026	
7.	Узгодження з керівником інтерфейсу користувача	05.04.2026	
8.	Розробка програмного забезпечення	10.04.2026	
9.	Налагодження розрахункової частини програми	15.04.2026	
10.	Розробка та налагодження інтерфейсної частини програми	20.04.2026	
11.	Узгодження з керівником набору тестів для контрольного прикладу	27.04.2026	
12.	Тестування програми	01.05.2026	
13.	Підготовка пояснювальної записки	05.05.2026	
14.	Здача курсової роботи на перевірку	10.05.2026	
15.	Захист курсової роботи	15.05.2026	

Студент _____
(підпис)

Керівник _____
(підпис)

Вітковська І.І.
(прізвище, ім'я, по батькові)

«10» травня 2026 р.

АНОТАЦІЯ

Пояснювальна записка до курсової роботи: 72 сторінки, 16 рисунків, 12 таблиць, 4 посилання.

Мета роботи: забезпечення коректної роботи програмного забезпечення, яке дозволяє здійснювати додавання, віднімання, множення методом Карацуби, ділення, піднесення до степеня і знаходження факторіалу довгих цілих чисел.

Вивчено методи арифметики довгих чисел; розроблено та реалізовано алгоритми довгого додавання, віднімання, множення методом Карацуби, ділення, піднесення до степеня та обчислення факторіалу.

Розроблено та реалізовано алгоритми зазначених методів.

Виконана програмна реалізація алгоритмів мовою C# у вигляді Windows Forms додатку.

ДОВГЕ ЧИСЛО, АРИФМЕТИКА ДОВГИХ ЧИСЕЛ, МЕТОД КАРАЦУБИ, ФАКТОРІАЛ, ПІДНЕСЕННЯ ДО СТЕПЕНЯ, ДОВГЕ ДІЛЕННЯ, BIGINT.

ЗМІСТ

ВСТУП.....	7
1 ПОСТАНОВКА ЗАДАЧІ.....	8
2 ТЕОРЕТИЧНІ ВІДОМОСТІ.....	9
2.1 Представлення довгих чисел у пам'яті	9
2.2 Алгоритм методу довгого додавання	9
2.3 Алгоритм методу довгого віднімання	9
2.4 Алгоритм стандартного довгого множення	10
2.5 Алгоритм методу довгого множення методом Карацуби	10
2.6 Алгоритм методу довгого ділення	11
2.7 Алгоритм методу піднесення довгого числа до ступеня.....	11
2.8 Алгоритм методу обчислення факторіалу довгого числа	12
3 ОПИС АЛГОРИТМІВ	13
3.1 Загальний алгоритм.....	13
3.2 Алгоритм методу довгого додавання	14
3.3 Алгоритм методу довгого віднімання	15
3.4 Алгоритм довгого множення методом Карацуби.....	16
3.5 Алгоритм методу довгого ділення	17
3.6 Алгоритм методу піднесення довгого числа до степеня	18
3.7 Алгоритм методу обчислення факторіалу довгого числа	18
4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	19
4.1 Діаграма класів програмного забезпечення	19
4.2 Опис методів частин програмного забезпечення	21
4.2.1 Стандартні методи	21
4.2.2 Користувацькі методи	23

5	ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	26
5.1	План тестування	26
5.2	Приклади тестування	27
6	ІНСТРУКЦІЯ КОРИСТУВАЧА	33
6.1	Робота з програмою	33
6.2	Формат вхідних та вихідних даних	36
6.3	Системні вимоги	36
7	АНАЛІЗ РЕЗУЛЬТАТІВ	38
	ВИСНОВКИ	47
	ПЕРЕЛІК ПОСИЛАНЬ	48
	ДОДАТОК А ТЕХНІЧНЕ ЗАВДАННЯ	49
	ДОДАТОК Б ТЕКСТИ ПРОГРАМНОГО КОДУ	52

ВСТУП

Сучасні комп'ютери оперують числами обмеженої розрядності – як правило, до 64 біт, що відповідає приблизно 18–19 десятковим цифрам. Проте у таких галузях, як криптографія, теорія чисел, наукові обчислення та комбінаторика, нерідко виникає потреба у роботі з числами, що містять сотні й тисячі цифр. Для вирішення цієї задачі розроблено методи так званої «арифметики довгих чисел» – набір алгоритмів, що дозволяють виконувати базові арифметичні операції над числами довільної розрядності.

У даній курсовій роботі розглядаються такі методи: довге додавання, довге віднімання, стандартне множення та метод Карацуби, довге ділення з остачею, швидке піднесення до степеня та обчислення факторіалу. Метод Карацуби є особливо важливим, оскільки він дозволяє знизити алгоритмічну складність множення з $O(n^2)$ до $O(n^{1.585})$, що суттєво прискорює обчислення для великих чисел.

У рамках роботи розроблено програмне забезпечення мовою C# з графічним інтерфейсом Windows Forms, що реалізує всі перелічені операції над цілими числами довжиною до 30 000 знаків. Програма також здійснює валідацію вхідних даних, обробку виняткових ситуацій та підрахунок кількості виконаних елементарних операцій для оцінки ефективності алгоритмів.

1 ПОСТАНОВКА ЗАДАЧІ

Розробити програмне забезпечення що буде реалізувати наступні арифметичні операції над довгими числами:

- а) Операція додавання
- б) Операція віднімання
- в) Операція множення методом Карацуби
- г) Операція ділення
- д) Операція піднесення до ступеня
- е) Операція обчислення факторіалу

Вхідними даними для програмного забезпечення є довгі цілі числа, які задаються користувачем. Програмне забезпечення повинно обробляти цілі числа довжиною до 30 000 знаків для операцій додавання, віднімання, множення та ділення; числа від 0 до 10 000 для обчислення факторіалу; а також пари чисел для піднесення до степеня, де результат не перевищує 60 000 знаків.

Вихідними даними для програмного забезпечення буде довге число, що є результатом виконання обраної арифметичної операції, яке виводиться на екран. Програмне забезпечення повинно видавати розв'язок за умови, що вхідні дані є цілими числами в заданому діапазоні. Якщо це не так, то програма повинна вивести відповідне повідомлення. У випадку ділення на нуль програма повинна вивести відповідне повідомлення про неможливість виконання цієї операції. У випадку спроби обчислення факторіала від від'ємного числа або піднесення у від'ємну степінь програма повинна вивести відповідне повідомлення про неможливість виконання цієї операції. У випадку, якщо введене користувачем число перевищує ліміт у 30000 знаків – програма виводить відповідне повідомлення. Програма повинна видавати повідомлення про помилки у випадку некоректних дій користувача та у випадку виникнення виняткових ситуацій.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1 Представлення довгих чисел у пам'яті

Довге ціле число зберігається у вигляді списку цілих чисел (`List<int>`), де кожен елемент є «цифрою» в системі числення з основою $\text{BASE} = 10^9$ (тобто блоком із 9 десяткових цифр). Молодший блок зберігається на початку списку. Знак числа зберігається окремо у булевій змінній `sign` (`true` — додатне, `false` — від'ємне). Таке подання дозволяє ефективно виконувати арифметичні операції, обробляючи число блоками по 9 цифр.

Наприклад, число 1 000 000 007 000 000 001 буде представлено як: [1, 7, 1] при $\text{BASE} = 10^9$.

2.2 Алгоритм методу довгого додавання

Процес додавання двох невід'ємних чисел великої розрядності концептуально повторює класичне ручне додавання у стовпчик. Числа розбиваються на блоки (цифри у системі числення з великою основою) і опрацьовуються послідовно, починаючи від наймолодших розрядів до найстарших.

Для кожної пари відповідних блоків обчислюється їхня сума, до якої додається значення перенесення з попереднього кроку. Якщо отримана сума дорівнює або перевищує основу системи числення, від неї віднімається значення основи, а перенесення для наступного старшого розряду встановлюється рівним одиниці. В іншому випадку перенесення дорівнює нулю. Якщо після проходження всіх блоків залишається ненульове перенесення, до результату додається новий старший блок. Часова складність алгоритму становить $O(n)$, де n – кількість блоків більшого з доданків.

2.3 Алгоритм методу довгого віднімання

Алгоритм віднімання чисел (за умови, що зменшуване більше або дорівнює від'ємнику) також базується на принципі ручного обчислення у стовпчик. Блоки чисел обробляються зліва направо.

На кожному етапі від поточного блоку зменшуваного віднімається відповідний блок від'ємника та значення «позики» з попереднього кроку. Якщо блок зменшуваного виявляється меншим за суму від'ємника та позики, до нього умовно додається значення основи системи числення, а індикатор позики для наступного кроку набуває значення одиниці. Якщо ж значення достатнє для віднімання, позика обнуляється. Знак кінцевого результату визначається окремо шляхом порівняння абсолютних значень початкових операндів. Складність даної операції становить $O(n)$, де n — кількість блоків більшого числа.

2.4 Алгоритм стандартного довгого множення

Стандартне множення реалізується за класичним принципом перемноження кожного блоку першого множника на кожен блок другого множника.

Отриманий добуток додається до відповідної позиції результату (яка визначається сумою індексів блоків множників), враховуючи перенесення від попередніх операцій. Новим значенням блоку результату на цій позиції стає остача від ділення поточної суми на основу системи числення, тоді як ціла частина від цього ділення переноситься на наступний старший розряд. Складність такого алгоритму є квадратичною — $O(n^2)$, тому його доцільно застосовувати лише для чисел відносно малої довжини (наприклад, до 10 блоків).

2.5 Алгоритм методу довгого множення методом Карацуби

Метод Карацуби є алгоритмом швидкого множення, який базується на парадигмі «розділяй і володарюй». Він дозволяє суттєво зменшити кількість необхідних елементарних операцій для чисел великої розрядності.

Два числа A та B , що складаються з n блоків, умовно розбиваються навпіл на дві частини розміром $m = n/2$. Їх можна математично подати як:

$$A = A_1 * \text{BASE}^m + A_0$$

$$B = B_1 * BASE^m + B_0$$

Замість чотирьох рекурсивних перемножень частин, алгоритм Карацуби обчислює результат за допомогою лише трьох проміжних добутоків:

$$P_1 = A_1 * B_1$$

$$P_2 = A_0 * B_0$$

$$P_3 = (A_1 + A_0) * (B_1 + B_0)$$

Кінцевий добуток формується за математичною формулою:

$$A * B = P_1 * BASE^{2m} + (P_3 - P_1 - P_2) * BASE^m + P_2$$

Такий підхід дозволяє знизити асимптотичну складність множення з квадратичної до $O(n^{\log_2 3}) \approx O(n^{1.585})$.

2.6 Алгоритм методу довгого ділення

Довге ділення з остачею реалізовано як ітераційний процес зі зсувами, під час якого частка формується поблочно.

На кожній ітерації алгоритм шукає найбільший цілий множник (у межах від нуля до основи системи числення мінус один), добуток якого з дільником не перевищує поточного залишку. Для оптимізації пошуку цього множника застосовується метод бінарного пошуку. Знайдена цифра додається до частки зі зсувом, а залишок оновлюється шляхом віднімання отриманого добутку. Цей процес повторюється доти, доки залишок не стане строго меншим за дільник. У гіршому випадку часова складність алгоритму становить $O(n^2 * \log BASE)$.

2.7 Алгоритм методу піднесення довгого числа до ступеня

Для виконання цієї операції застосовується метод швидкого бінарного піднесення до степеня, що дозволяє уникнути багаторазового лінійного множення.

Процес полягає у послідовному піднесенні основи до квадрата та двократному зменшенні показника степеня. Якщо на певній ітерації поточний

показник степеня є непарним числом, загальний результат додатково домножується на поточне значення основи. Ітерації тривають, доки показник степеня не досягне нуля. Складність цього методу становить $O(\log x)$ операцій множення довгих чисел (де x – показник степеня), що робить його вкрай ефективним для великих значень.

2.8 Алгоритм методу обчислення факторіалу довгого числа

Факторіал заданого числа обчислюється шляхом послідовного математичного множення натурального ряду від заданого значення n до одиниці.

На кожному кроці поточний акумульований результат перемножується на поточне ціле значення, після чого множник зменшується на одиницю. Складність алгоритму становить $O(n)$ множень. Враховуючи, що значення факторіалу зростає надзвичайно стрімко, для практичних обчислень у рамках програмного забезпечення вводиться обмеження на максимальне значення аргументу (наприклад, до 10 000).

3 ОПИС АЛГОРИТМІВ

Перелік всіх основних змінних та їхнє призначення наведено в таблиці

3.1.

Таблиця 3.1 – Основні змінні та їхні призначення

Змінна	Призначення
BASE	Основа системи числення ($10^9 = 1\,000\,000\,000$)
ChunkSize	Кількість десяткових цифр в одному блоці (9)
Steps	Лічильник практичної складності алгоритму
myBigInt	Зберігає значення числа
sign	Знак числа
MyBigIntLength	Кількість блоків числа
savedNumber	Введене користувачем число
secondNumber	Друге введене користувачем число (у випадку вибору бінарних операції)
currentOperation	Зберігає обрану користувачем арифметичну операцію
carry	Перенос при додаванні / позика при відніманні
result	Проміжний або кінцевий результат обчислення
remainder	Остача від ділення

3.1 Загальний алгоритм

1. ПОЧАТОК
2. Зчитати перше число (savedNumber) з поля TextLine.
 - 2.1. ЯКЩО введені дані містять некоректні символи ТО видати повідомлення про помилку та перейти до пункту 14
3. Зчитати обрану операцію (currentOperation).
4. ЯКЩО обрана операція обчислення факторіалу:

- 4.1. ЯКЩО число перевищує 10 000, ТО видати повідомлення про помилку та перейти до пункту 14.
- 4.2. Обробити дані згідно алгоритму обчислення факторіалу довгого числа (підрозділ 3.7) та перейти до пункту 12.
5. Зчитати друге число (secondNumber) з поля TextLine.
6. ЯКЩО введені дані містять некоректні символи, ТО видати повідомлення про помилку та перейти до пункту 14.
7. ЯКЩО обрана операція додавання, ТО обробити дані згідно алгоритму довгого додавання (підрозділ 3.2)
8. ЯКЩО обрана операція віднімання, ТО обробити дані згідно алгоритму довгого віднімання (підрозділ 3.3)
9. ЯКЩО обрана операція множення, ТО обробити дані згідно алгоритму довгого множення методом Карацуби (підрозділ 3.4)
10. ЯКЩО обрана операція ділення:
 - 10.1. ЯКЩО другий операнд дорівнює нулю, ТО видати повідомлення про помилку та перейти до пункту 14.
ІНАКШЕ обробити дані згідно алгоритму довгого ділення (підрозділ 3.5).
11. ЯКЩО обрана операція піднесення до степеня:
 - 11.1. ЯКЩО степінь від'ємна ТО видати повідомлення про помилку та перейти до пункту 14.
 - 11.2. ЯКЩО результат перевищує 60 000 знаків видати повідомлення про помилку та перейти до пункту 14.
ІНАКШЕ обробити дані згідно алгоритму піднесення довгого числа до ступеня (підрозділ 3.6).
12. Вивести результат на екран.
13. Вивести кількість операцій
14. КІНЕЦЬ

3.2 Алгоритм методу довгого додавання

1. ПОЧАТОК
2. Вхідні дані: масиви блоків абсолютних значень чисел a та b .
3. Ініціалізація змінних $carry = 0$ та $i = 0$.
4. Обчислити $maxSize$ = максимальна довжина між масивами a та b .
5. Ініціалізація порожнього числа $result$.
6. ЦИКЛ ПОКИ $i < maxSize$ АБО $carry \neq 0$:
 - 5.1. Обчислити $temp = a[i] + b[i] + carry$ (якщо індекс виходить за межі довжини масиву, відповідне значення вважати рівним 0).
 - 5.2. ЯКЩО $temp \geq BASE$, ТО:
 - 5.2.1. Додати до масиву $result$ значення $temp - BASE$.
 - 5.2.2. Присвоїти $carry = 1$.
 - 5.3. ІНАКШЕ Додати до масиву $result$ значення $temp$.
 - 5.3.1. Присвоїти $carry = 0$.
 - 5.3.2. Збільшити i на 1.
7. Виконати нормалізацію числа $result$ (видалити нулі наприкінці масиву, що відповідають старшим розрядам).
8. Встановити знак результату згідно зі знаком початкових чисел.
9. КІНЕЦЬ

3.3 Алгоритм методу довгого віднімання

1. ПОЧАТОК
2. Вхідні дані: масиви блоків абсолютних значень чисел a та b (за умови, що $a \geq b$).
3. Ініціалізація змінних: $carry = 0$, $i = 0$.
4. Обчислити $maxSize$ = максимальна довжина між масивами a та b .
5. Ініціалізація порожнього числа $result$.
6. ЦИКЛ ПОКИ $i < maxSize$ АБО $carry \neq 0$:
 - 6.1. Обчислити $temp = b[i] + carry$ (якщо індекс виходить за межі масиву, значення вважати рівним 0).
 - 6.2. ЯКЩО $a[i] < temp$, ТО:

6.2.1. Додати до масиву result значення $BASE + a[i] - temp$.

6.2.2. Присвоїти $carry = 1$.

6.3. ІНАКШЕ Додати до масиву result значення $a[i] - temp$.

6.3.1. Присвоїти $carry = 0$.

6.3.2. Збільшити i на 1.

7. Виконати нормалізацію числа result (видалення зайвих нулів старших розрядів).

8. КІНЕЦЬ

3.4 Алгоритм довгого множення методом Карацуби

1. ПОЧАТОК

2. Вхідні дані: числа a та b .

3. ЯКЩО довжина масиву $a < 10$ АБО довжина масиву $b < 10$, ТО виконати стандартне множення «в стовпчик» та повернути результат.

4. Знайти $n = \text{максимальна довжина між масивами } a \text{ та } b$.

5. Обчислити $m = n / 2$.

6. Розділити масив числа a на дві частини: a_1 (старші блоки від m до кінця) та a_0 (молодші блоки від 0 до m).

7. Розділити масив числа b на дві частини: b_1 та b_0 аналогічним чином.

8. Обчислити рекурсивно $p_1 = a_1 * b_1$.

9. Обчислити рекурсивно $p_2 = a_0 * b_0$.

10. Обчислити рекурсивно $p_3 = (a_1 + a_0) * (b_1 + b_0) - p_1 - p_2$.

11. Обчислити $result = p_1$, зсунуте вліво на $2*m$ блоків + p_3 , зсунуте вліво на m блоків + p_2 .

12. ЯКЩО $result \neq 0$, ТО встановити знак результату.

12.1. ЯКЩО знаки a та b однакові, встановити знак результату як додатній.

12.2. ІНАКШЕ встановити знак результату як від'ємний.

13. КІНЕЦЬ

3.5 Алгоритм методу довгого ділення

1. ПОЧАТОК
2. Вхідні дані: ділене a , дільник b .
3. ЯКЩО $b = 0$, ТО згенерувати виняткову ситуацію "Ділення на нуль неможливе".
4. Ініціалізація абсолютних значень: $absA = |a|$, $absB = |b|$.
5. Ініціалізація змінних: $q = absA$, $shift = (\text{довжина } absA - \text{довжина } absB)$, $p = 0$, $aLength = \text{довжина } q$.
6. ЦИКЛ ПОКИ $absB \leq q$:
 - 6.1. Присвоїти $o = 1$.
 - 6.2. Обчислити $temp = q$, зсунуте вправо на $shift$ блоків.
 - 6.3. ЯКЩО $temp < absB$, ТО зменшити $shift$ на 1.
 - 6.3.1 Оновити $temp = q$, зсунуте вправо на $shift$ блоків.
 - 6.3.2 Присвоїти $o = 0$.
 - 6.4. Знайти множник d за допомогою бінарного пошуку (найбільше ціле число, для якого $absB * d \leq temp$).
 - 6.5. Обчислити $m = absB * d$.
 - 6.6. Оновити $q = q - (m, \text{зсунуте вліво на } shift \text{ блоків})$.
 - 6.7. Оновити $p = (p, \text{зсунуте вліво на } 1 \text{ блок}) + d$.
 - 6.8. ЯКЩО $q < absB$, ТО $p = p$, зсунуте вліво на $shift$ блоків.
 - 6.9. ІНАКШЕ $p = p$, зсунуте вліво на $(aLength - \text{довжина } q - o - 1)$ блоків.
 - 6.10. Оновити $aLength = \text{довжина } q$.
7. Встановити остачу $remainder = q$.
8. ЯКЩО $remainder \neq 0$, ТО встановити знак остачі рівним знаку числа a .
9. ЯКЩО $p \neq 0$, ТО встановити знак результату p
 - 9.1. ЯКЩО знаки a та b однакові, ТО встановити знак результату p як додатній.

9.2. ІНАКШЕ встановити знак результату р як від'ємний.

10. Повернути р.

11. КІНЕЦЬ

3.6 Алгоритм методу піднесення довгого числа до степеня

1. ПОЧАТОК

2. Вхідні дані: основа number, степінь x (ціле число).

3. ЯКЩО $x < 0$, ТО згенерувати виняткову ситуацію "Степінь не може бути від'ємним".

4. ЯКЩО $x = 0$, ТО повернути 1.

5. ЯКЩО number = 0 АБО number = 1, ТО повернути number.

6. Ініціалізація змінних: result = 1, currentBase = number.

7. ЦИКЛ ПОКИ $x > 0$:

7.1. ЯКЩО x непарне, ТО присвоїти result = result * currentBase.

7.2. Присвоїти currentBase = currentBase * currentBase.

7.3. Розділити x на 2.

8. Повернути result.

9. КІНЕЦЬ

3.7 Алгоритм методу обчислення факторіалу довгого числа

1. ПОЧАТОК

2. Вхідні дані: число number.

3. ЯКЩО number < 0, ТО згенерувати виняткову ситуацію "Неможливо обчислити факторіал від'ємного числа".

4. ЯКЩО number = 0 АБО number = 1, ТО повернути 1.

5. Ініціалізація змінних result = 1 та temp = number.

6. ЦИКЛ ПОКИ temp > 1:

6.1. Присвоїти result = result * temp.

6.2. Зменшити temp на 1.

7. Повернути result.

8. КІНЕЦЬ

4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Діаграма класів програмного забезпечення

Розроблене програмне забезпечення побудоване з використанням принципів об'єктно-орієнтованого програмування та розділене на логічні модулі. Архітектура додатку складається з п'яти основних компонентів, взаємодія між якими відображена на діаграмі класів (Рисунок 4.1).

Основу предметної області складає клас `BigInt`, який інкапсулює логіку зберігання довгих цілих чисел (за допомогою динамічного списку блоків `myBigInt`) та їх знак. У цьому класі перевизначено базові математичні оператори (+, -, *, /, %) та реалізовано алгоритми довгої арифметики, зокрема метод множення Карацуби. Клас `BigInt` функціонує як незалежний тип даних.

Для реалізації більш складних математичних алгоритмів виділено статичний клас `BigMath`. Він містить методи обчислення факторіалу (`Factorial`) та швидкого піднесення до степеня (`BigPow`). Цей клас має пряму залежність від класу `BigInt`, оскільки оперує його екземплярами – приймає їх у якості вхідних параметрів та повертає як результат обчислень.

Для забезпечення можливості аналізу ефективності алгоритмів реалізовано допоміжний статичний клас `OpCounter`. Він містить глобальний лічильник `Steps`. Класи `BigInt` та `BigMath` залежать від нього: під час виконання внутрішніх циклів та рекурсивних викликів вони звертаються до методу `OpCounter.Add()`, фіксуючи кожну елементарну обчислювальну операцію.

Рівень графічного інтерфейсу (`Presentation Layer`) представлено класом `Form1`, який успадковується від стандартного класу `System.Windows.Forms.Form`. `Form1` керує взаємодією з користувачем: зчитує ввід, обробляє події натискання кнопок та виводить результати. Між `Form1` та `BigInt` існує зв'язок типу спрямованої асоціації (`Directed Association`), оскільки форма містить приватні поля `savedNumber` типу `BigInt` для збереження

проміжних результатів. Також форма залежить від класу `BigMath` для виконання відповідних унарних операцій та від класу `OpCounter` для зчитування і скидання лічильника складності перед кожним новим обчисленням.

Точкою входу в програму є статичний клас `Program`, головний метод якого (`Main`) ініціалізує та запускає відображення головної форми `Form1`.

Така архітектура з чітким розподілом відповідальності (відокремлення математичної логіки від інтерфейсу користувача) забезпечує високу надійність програмного забезпечення, легкість його тестування та можливість подальшого розширення новими математичними функціями без зміни існуючого інтерфейсу.

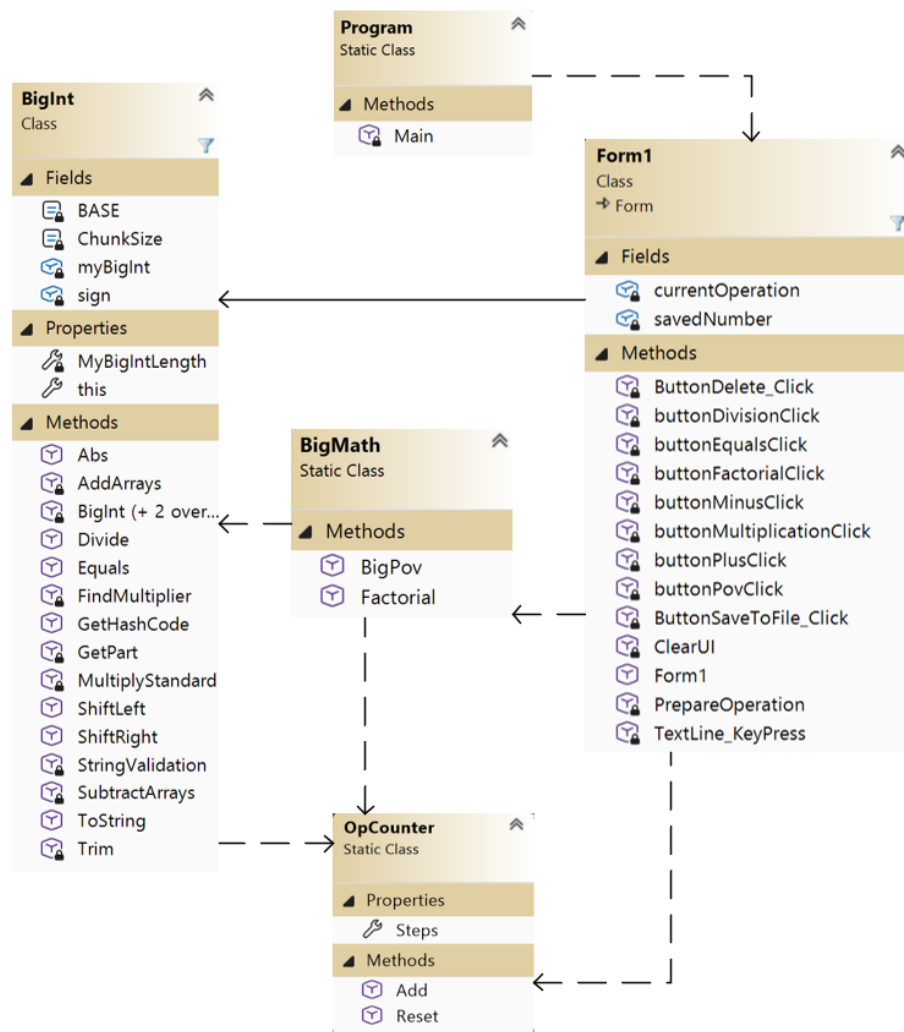


Рисунок 0.1 – Діаграма класів

4.2 Опис методів частин програмного забезпечення

4.2.1 Стандартні методи

У таблиці 4.1 наведено стандартні методи .NET, що використовуються в програмному забезпеченні.

Таблиця 4.1 – Стандартні методи

№ п/ п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметр ів	Опис вихідних параметр ів	Заголовний файл
1	List<int>	Add(int)	Додавання елементу до списку	int item	void	System.Collections.Generic
2	List<int>	RemoveAt(int)	Видалення елементу за індексом	int index	void	System.Collections.Generic
3	string	StartsWith(string)	Перевіряє початок рядка	string value	bool	System
4	int	TryParse(string, out int)	Перетворення рядка на ціле число	string s, out int result	bool	System
5	Math	Abs(long)	Абсолютне значення числа	long value	long	System
6	Math	Max(int, int)	Максимум двох чисел	int a, int b	int	System
7	Math	Min(int, int)	Мінімум двох значень	int a, int b	int	System

Продовження таблиці 4.1

№ п/ п	Назва класу	Назва функції	Призначе ння функції	Опис вхідних парамет рів	Опис вихідних параметрів	Заголовний файл
8	MessageBox	Show(string, string)	Виведенн я діалоговог о вікна	string text, string caption	DialogResult	System.Windows. Forms
9	File	WriteAllText(s tring, string)	Запис рядка у файл	string path, string contents	void	System.IO
10	string	Substring(int, int)	Отримує підрядок	int start, int length	string	System
11	StringBuil der	Append(string)	Додає рядок до білдера	string value	StringBuilde r	System.Text
12	Enumerab le	Repeat<T>(T, int)	Генерує послідовн ість повторень	T element, int count	IEnumerable <T>	System.Linq

4.2.2 Користувацькі методи

У таблиці 4.2 наведено основні користувацькі методи, розроблені в межах курсової роботи.

Таблиця 4.2 – Користувацькі методи

№ п/п	Назва класу	Назва функції	Призначенн я функції	Опис вхідних параметрів	Опис вихідних параметрі в	Заголовний файл
1	BigInt	Divide	Ділення з остачею	BigInt a, BigInt b, out BigInt remainder	p (частка)	Kyrsova_2_sem
2	BigInt	AddArrays	Додавання абсолютних значень	BigInt a, BigInt b	result	Kyrsova_2_sem
3	BigInt	SubtractArrays	Віднімання абсолютних значень	BigInt a, BigInt b	result	Kyrsova_2_sem
4	BigInt	StringValidation	Валідація та розбиття рядка на блоки	string input, out List<int> myBigInt	true/ false	Kyrsova_2_sem
5	BigInt	MultiplyStandard	Стандартне множення «в стовпчик»	BigInt a, BigInt b	result	Kyrsova_2_sem
6	BigInt	FindMultiplier	Бінарний пошук множника для ділення	BigInt b, BigInt temp	bestMultipl ier	Kyrsova_2_sem
7	BigMath	Factorial	Обчислення факторіалу	BigInt number	result	Kyrsova_2_sem

Продовження таблиці 4.2

№ п/п	Назва класу	Назва функції	Призначенн я функції	Опис вхідних параметрів	Опис вихідних параметрі в	Заголовний файл
8	BigMath	BigPov	Піднесення до цілого степеня	BigInt number, int x	result	Kyrsova_2_sem
9	OpCoun ter	Reset	Скидання лічильника операцій	void	void	Kyrsova_2_sem
10	OpCoun ter	Add	Збільшення лічильника операцій	long count = 1	void	Kyrsova_2_sem
11	BigInt	Trim()	Видаляє зайві нульові старші блоки	void	void	Kyrsova_2_sem
12	BigInt	operator *	Множення методом Карацуби	BigInt a, BigInt b	BigInt	Kyrsova_2_sem
13	BigInt	operator +	Оператор додавання з урахуванням знаку	BigInt a, BigInt b	BigInt	Kyrsova_2_sem
14	BigInt	operator -	Оператор віднімання з урахуванням знаку	BigInt a, BigInt b	BigInt	Kyrsova_2_sem
15	BigInt	operator /	Оператор цілочисельн ого ділення	BigInt a, BigInt b	BigInt	Kyrsova_2_sem

Продовження таблиці 4.2

№ п/п	Назва класу	Назва функції	Призначенн я функції	Опис вхідних параметрів	Опис вихідних параметрі в	Заголовний файл
16	BigInt	operator %	Оператор остачі від ділення	BigInt a, BigInt b	BigInt	Kyrsova_2_sem
17	BigInt	ToString()	Перетворенн я BigInt у рядок	void	string	Kyrsova_2_sem
18	BigInt	operator <, >, <=, >=, ==, !=	Оператори порівняння двох BigInt	BigInt a, BigInt b	bool	Kyrsova_2_sem

5 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 План тестування

Тестування програмного забезпечення виконується з метою перевірки коректності роботи всіх реалізованих методів та стійкості програми до некоректних дій користувача. Тестування проводиться за наступним планом:

- а) Тестування правильності введених значень.
 - 1) Тестування при введенні некоректних символів.
 - 2) Тестування при введенні числа, що перевищує ліміт 30 000 знаків.
 - 3) Тестування при введенні порожнього рядка.
- б) Тестування коректної роботи методів при виключних випадках.
 - 1) Тестування при спробі ділення на нуль.
 - 2) Тестування при введенні від'ємного значення ступеня.
 - 3) Тестування при спробі обчислення факторіалу від від'ємного числа.
 - 4) Тестування при спробі обчислення факторіалу числа, що перевищує 10 000.
 - 5) Тестування при спробі піднесення, якщо результат перевищує 60 000 знаків.
- в) Тестування коректності роботи методів довгого додавання, віднімання, множення методом Карацуби, ділення, піднесення до ступеня, обчислення факторіалу при додатніх значеннях.
 - 1) Перевірка коректності роботи методу довгого додавання.
 - 2) Перевірка коректності роботи методу довгого віднімання.
 - 3) Перевірка коректності роботи методу довгого множення методом Карацуби.
 - 4) Перевірка коректності роботи методу довгого ділення.
 - 5) Перевірка коректності роботи методу піднесення до ступеня довгого числа.

6) Перевірка коректності роботи методу обчислення факторіалу довгого числа.

г) Тестування коректності роботи методів довгого додавання, віднімання, множення методом Карацуби, ділення, піднесення до ступеня, обчислення факторіалу при від'ємних значеннях.

1) Перевірка коректності роботи методу довгого додавання.

2) Перевірка коректності роботи методу довгого віднімання.

3) Перевірка коректності роботи методу довгого множення методом Карацуби.

4) Перевірка коректності роботи методу довгого ділення.

5) Перевірка коректності роботи методу піднесення до ступеня довгого числа.

д) Тестування ефективності алгоритмів.

1) Порівняння кількості операцій для різних розмірів вхідних чисел.

5.2 Приклади тестування

У цьому розділі наведено приклади тестування програмного забезпечення, розробленого для виконання арифметичних операцій над цілими числами довільної розрядності, згідно з планом тестування, описаним у розділі 5.1.

Тестування проводиться для перевірки стійкості та коректності роботи програми в різних сценаріях: при введенні некоректних символів (літер або спеціальних знаків), перевищенні встановлених лімітів кількості знаків (понад 30 000), спробах виконання недопустимих операцій, таких як ділення на нуль, обчислення факторіалу від'ємного числа або піднесення у від'ємну степінь. Також перевіряється точність реалізації основних алгоритмів: довгого додавання та віднімання, множення методом Карацуби, ділення з остачею, швидкого піднесення до ступеня та знаходження факторіалу.

Кожен приклад тесту містить визначену мету, початковий стан інтерфейсу, конкретні вхідні дані, детальну схему проведення випробування, а також порівняння очікуваного результату з фактичним станом програми після виконання операції. Результати задокументовані у формі таблиць, що дозволяє наочно підтвердити відповідність роботи програмного забезпечення технічним вимогам.

Нижче представлено описи тестів для ключових функціональних можливостей програми.

Таблиця 5.1 – Приклад роботи програми при введенні некоректних символів

Мета тесту	Перевірити можливість введення некоректних даних
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Bcr S-r u@г
Схема проведення тесту	Введення рядка з буквами в поле TextLine
Очікуваний результат	Символи не відображаються у полі вводу (заблоковані обробником KeyPress)
Стан програми після проведення випробувань	Символи не введено; поле залишається незмінним

Таблиця 5.2 – Приклад роботи програми при числа розміром більше за 30000 символів

Мета тесту	Перевірити можливість введення некоректних даних
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Число що містить 30001 символ
Схема проведення тесту	Введення числа в поле TextLine

Продовження таблиці 5.2

Очікуваний результат	Повідомлення про некоректний формат введених даних
Стан програми після проведення випробувань	Виведено повідомлення про помилку, програма очікує коректний ввід.

Таблиця 5.3 – Приклад роботи програми при відсутності вводу

Мета тесту	Перевірити можливість введення некоректних даних
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Відсутні
Схема проведення тесту	Відсутність вводу в поле TextLine
Очікуваний результат	Повідомлення про відсутність вводу і неможливість виконання арифметичної операції
Стан програми після проведення випробувань	Виведено повідомлення про помилку, програма очікує коректний ввід.

Таблиця 5.4 – Приклад роботи програми при спробі ділення на нуль.

Мета тесту	Перевірити обробку ділення на нуль
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Числа 123 і 0, операція «/»
Схема проведення тесту	1. Введення першого числа 123 в поле TextLine 2. Вибір операції «/» 3. Введення числа 0 в поле TextLine.

Продовження таблиці 5.4

Очікуваний результат	Повідомлення про неможливість виконання цієї операції
Стан програми після проведення випробувань	Виведено повідомлення про помилку, програма очікує коректний ввід дільника.

Таблиця 5.5 – Приклад роботи програми при введенні від’ємного ступеня

Мета тесту	Перевірити обробку від’ємного ступеня
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Числа 5 і -3, операція «^»
Схема проведення тесту	1. Введення першого числа 5 в поле TextLine 2. Вибір операції «^» 3. Введення числа -3 в поле TextLine.
Очікуваний результат	Повідомлення про неможливість виконання цієї операції
Стан програми після проведення випробувань	Виведено повідомлення про помилку, програма очікує коректний ввід ступеня.

Таблиця 5.6 – Збереження результатів у файл

Мета тесту	Перевірити можливість збереження результатів роботи програми у файл
Початковий стан програми	Відкрите вікно програми
Вхідні дані	Числа 10 і 37, операція «+»

Продовження таблиці 5.6

Схема проведення тесту	1. Запустити програму. 2. Ввести числа, обрати арифметичну операцію та обчислити розв'язок. 3. Натиснути кнопку «SAVE TO FILE». 4. Обрати місце збереження файлу. 5. Підтвердити збереження.
Очікуваний результат	Успішне збереження результатів виконання програми у файл
Стан програми після проведення випробувань	Програма успішно зберігає результати розв'язання у файл та виводить відповідне повідомлення.

У результаті проведеного тестування програмного забезпечення було перевірено ключові аспекти його функціонування, зокрема коректність введення великих чисел, надійність обробки помилок, правильність реалізації алгоритмів арифметики довгих чисел (додавання, віднімання, множення методом Карацуби, ділення, піднесення до степеня та обчислення факторіалу), а також коректність підрахунку кількості операцій та можливість збереження результатів у файл.

Під час випробувань встановлено, що програма ефективно обробляє некоректні вхідні дані, автоматично блокуючи введення нечислових символів. Програмне забезпечення правильно ідентифікує та сповіщає про виняткові ситуації, такі як спроба ділення на нуль, обчислення факторіалу від'ємного числа або піднесення у від'ємну степінь. Реалізовані алгоритми забезпечують точне обчислення результатів для цілих чисел довжиною до 30 000 знаків.

Окремо було підтверджено ефективність методу Карацуби для множення великих масивів даних та точність роботи алгоритму швидкого піднесення до степеня. Лічильник практичної складності коректно відображає кількість виконаних елементарних операцій, що дозволяє оцінити продуктивність програми. Функція експорту даних забезпечує надійне збереження введених чисел, обраної операції та отриманого результату в текстовий файл.

Таким чином, на основі результатів тестування можна зробити висновок, що розроблене програмне забезпечення для арифметики довгих чисел працює коректно, демонструє стабільність при великих навантаженнях та повністю відповідає вимогам технічного завдання.

6 ІНСТРУКЦІЯ КОРИСТУВАЧА

6.1 Робота з програмою

Після запуску виконавчого файлу з розширенням *.exe, відкривається головне вікно програми (Рисунок 6.1).

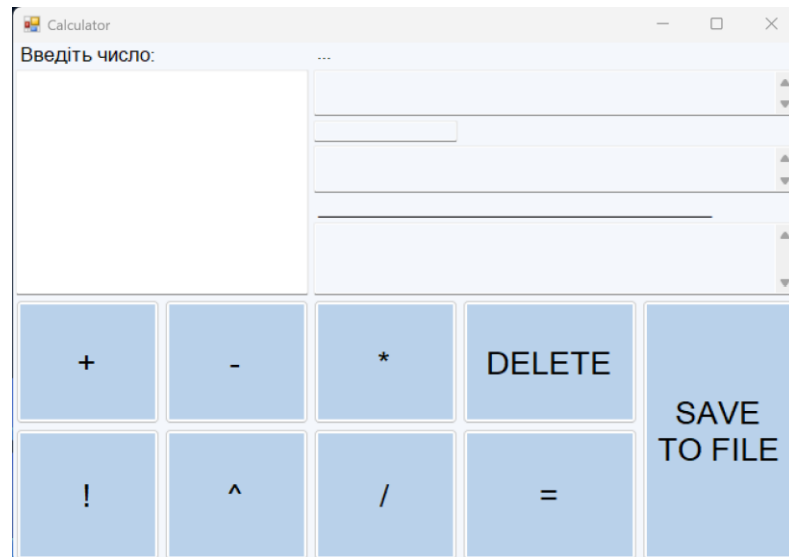


Рисунок 6.1 – Головне вікно програми

У лівій частині вікна розташоване поле введення числа. Для початку роботи необхідно ввести перше число за допомогою клавіатури (рисунок 6.2). Поле приймає лише цифри та знак мінус на початку – введення будь-яких інших символів автоматично блокується.

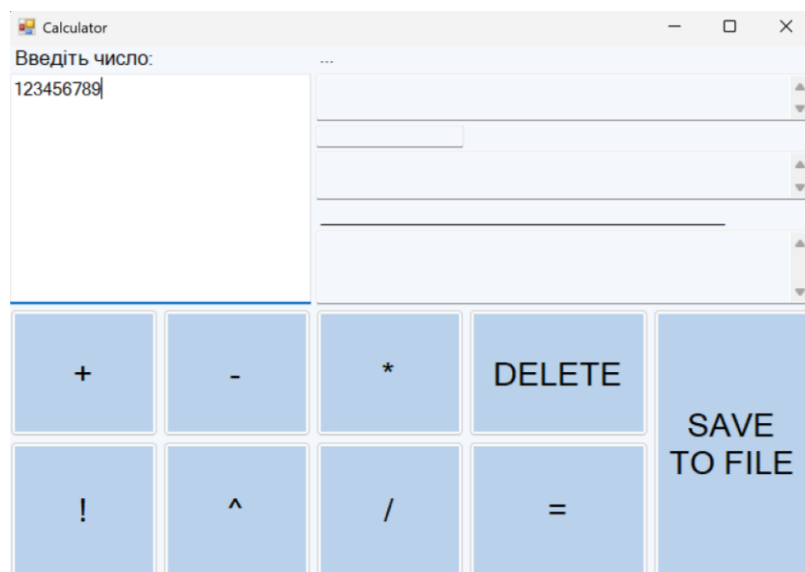


Рисунок 6.2 – Ввід першого числа

Після цього необхідно обрати арифметичну операцію шляхом натискання на відповідну клавішу у вікні програми: «+» - додавання, «-» - віднімання, «*» - множення, «/» - ділення, «!» - обчислення факторіалу, «^» - піднесення до ступеня. Якщо користувачем була обрана бінарна операція, необхідно ввести друге число для виконання обчислень (рисунок 6.3):

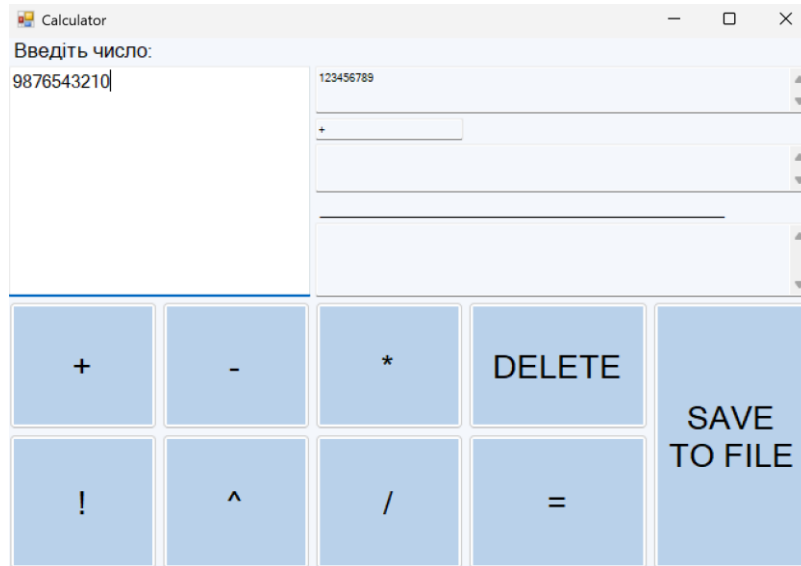


Рисунок 6.3 – Ввід другого числа (за умови обрання бінарної арифметичної операції)

Для відображення результатів обчислень необхідно натиснути клавішу «=». (рисунок 6.4):

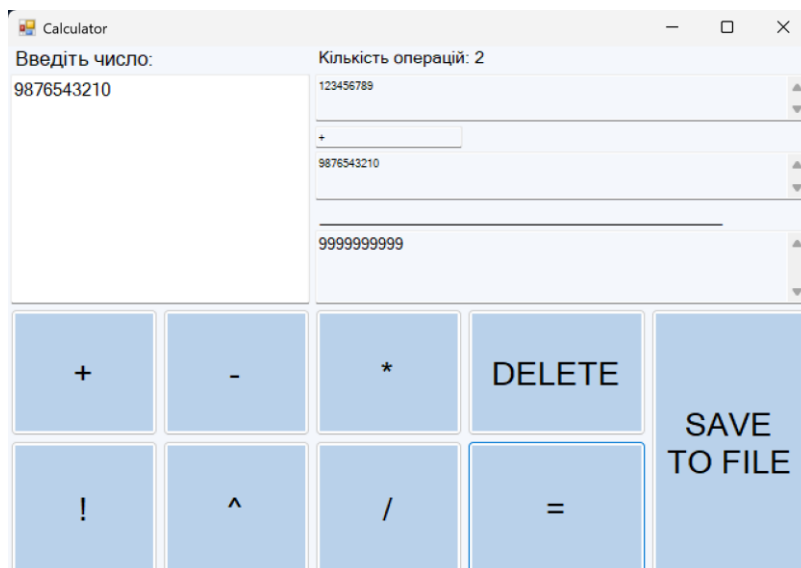


Рисунок 6.4 – Відображення результатів обчислень (бінарна операція)

Якщо користувачем була обрана унарна операція результати обчислень виведуться автоматично без натискання клавіші «=» (рисунок 6.5).

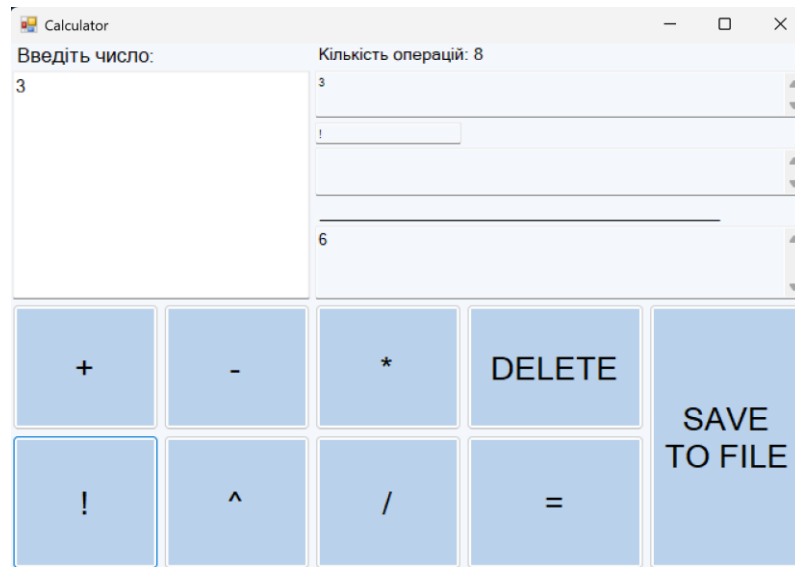


Рисунок 6.5 – Відображення результатів обчислень (унарна операція)

У нижній частині вікна відображається кількість виконаних операцій, що дозволяє оцінити практичну складність алгоритму.

Для збереження результатів обчислень у файл необхідно натиснути на клавішу «>», ввести назву файлу та обрати місце його зберігання. У файл записуються: перше число, операція, друге число (якщо воно є), результат та кількість операцій.

Для очищення полів вводу необхідно натиснути на клавішу «DELETE» (рисунок 6.6).

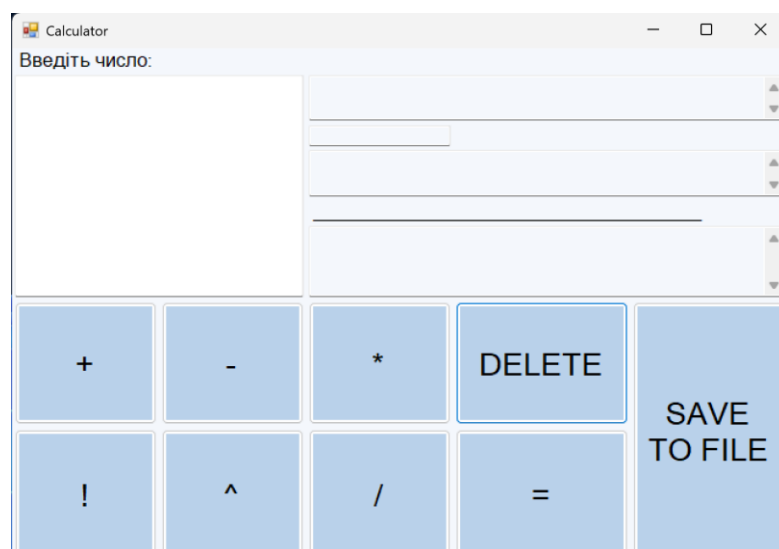


Рисунок 6.6 – Очищення полів вводу

6.2 Формат вхідних та вихідних даних

Користувачем на вхід програми подаються:

- для операцій додавання, віднімання, множення та ділення: два цілих числа довжиною не більше 30000 знаків;
- для операції обчислення факторіалу: ціле число в межах від 0 до 10000;
- для обчислення ступеня числа: ціла основа та цілий невід'ємний показник степеня, причому довжина результату не повинна перевищувати 60000 знаків

Результатом виконання програми є результат обчислення заданої арифметичної операції над вказаними значеннями.

6.3 Системні вимоги

Системні вимоги до програмного забезпечення наведені в таблиці 6.1.

Таблиця 6.1 – Системні вимоги програмного забезпечення

	Мінімальні	Рекомендовані
Операційна система	Windows XP / Vista / 7 / 8 / 10 / 11 (з останніми оновленнями)	Windows 10 / 11 (з останніми оновленнями)
Процесор	Intel Pentium III 1.0 GHz або AMD Athlon 1.0 GHz	Intel Core i3 або AMD Ryzen 3 та вище
Оперативна пам'ять	256 MB (XP) / 1 GB (7/8/10)	4 GB
Відеоадаптер	Intel GMA 950 з відеопам'яттю об'ємом не менше 64 МБ (або сумісний аналог)	
Дисплей	800x600	1024x768 або краще

Продовження таблиці 6.1

Прилади введення	Клавіатура, комп'ютерна миша	Клавіатура, комп'ютерна миша
Додаткове програмне забезпечення	Microsoft .Net Framework 4.7.2 або вище	Microsoft .Net Framework 4.7.2 або вище

7 АНАЛІЗ РЕЗУЛЬТАТІВ

Головною задачею курсової роботи була реалізація програми для виконання арифметичних операцій над довгими цілими числами. Усі шість операцій — додавання, віднімання, множення методом Карацуби, ділення, піднесення до степеня та обчислення факторіалу – реалізовані та протестовані.

Критичних ситуацій у роботі програми виявлено не було. Під час тестування встановлено, що всі виняткові ситуації (ділення на нуль, від'ємний степінь, від'ємний аргумент факторіалу, перевищення лімітів) обробляються коректно з відображенням відповідних повідомлень.

Для перевірки достовірності результатів використовувалися WolframAlpha. Всі незалежно обчислені результати збігаються з результатами виконання програми, що підтверджує правильність обчислень.

Результат додавання двох довгих чисел показано на рисунку 7.1.

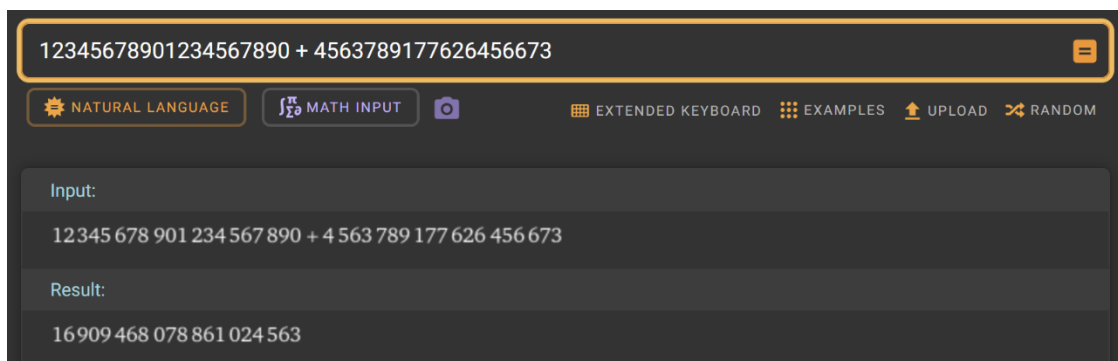


Рисунок 7.1 – Результат додавання двох довгих чисел.

Результат віднімання двох довгих чисел показано на рисунку 7.2.

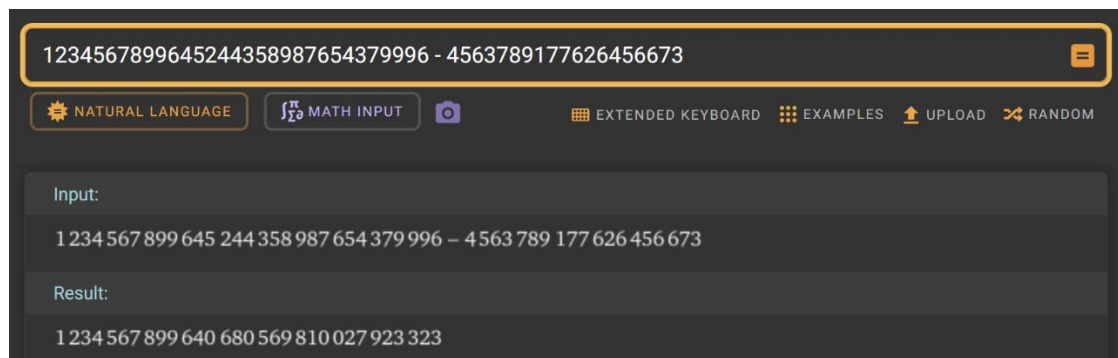


Рисунок 7.2 – Результат віднімання двох довгих чисел.

Результат множення двох довгих чисел показано на рисунку 7.3.

The screenshot shows a math calculator interface. At the top, the input field contains the expression $3456788253097524 * 6576752133255730065$. Below the input field, there are several buttons: "NATURAL LANGUAGE", "MATH INPUT", a camera icon, "EXTENDED KEYBOARD", "EXAMPLES", "UPLOAD", and "RANDOM". The "Input:" section displays the same expression. The "Result:" section shows the product: $22734439517772489508674558263859060$.

Рисунок 7.3 – Результат множення двох довгих чисел.

Результат ділення двох довгих чисел показано на рисунку 7.4.

The screenshot shows a math calculator interface. At the top, the input field contains the expression $466758698265453423567 / 76254545365$. Below the input field, there are several buttons: "NATURAL LANGUAGE", "MATH INPUT", a camera icon, "EXTENDED KEYBOARD", "EXAMPLES", "UPLOAD", and "RANDOM". The "Input:" section displays the division expression. The "Exact result:" section shows the fraction $\frac{466758698265453423567}{76254545365}$ with the note "(irreducible)". The "Mixed fraction:" section shows the mixed fraction $6121060666 \frac{28039310477}{76254545365}$. The "Quotient and remainder:" section shows the equation $466758698265453423567 = 6121060666 \times 76254545365 + 28039310477$. There are also buttons for "Step-by-step solution" and "Show details".

Рисунок 7.4 – Результат ділення двох довгих чисел.

Результат піднесення довгого числа до натурального показано на рисунку 7.5.

The screenshot shows a math calculator interface. At the top, the input field contains the expression 3443232^{23} . Below the input field, there are several buttons: "NATURAL LANGUAGE", "MATH INPUT", a camera icon, "EXTENDED KEYBOARD", "EXAMPLES", "UPLOAD", and "RANDOM". The "Input:" section displays the expression 3443232^{23} . The "Result:" section shows the result: $2239879082238317257739135943521550026977747612498009883183 \dots$. There is a button for "Step-by-step solution".

Рисунок 7.5 – Результат піднесення ступеня.

Результат обчислення факторіалу показано на рисунку 7.6.

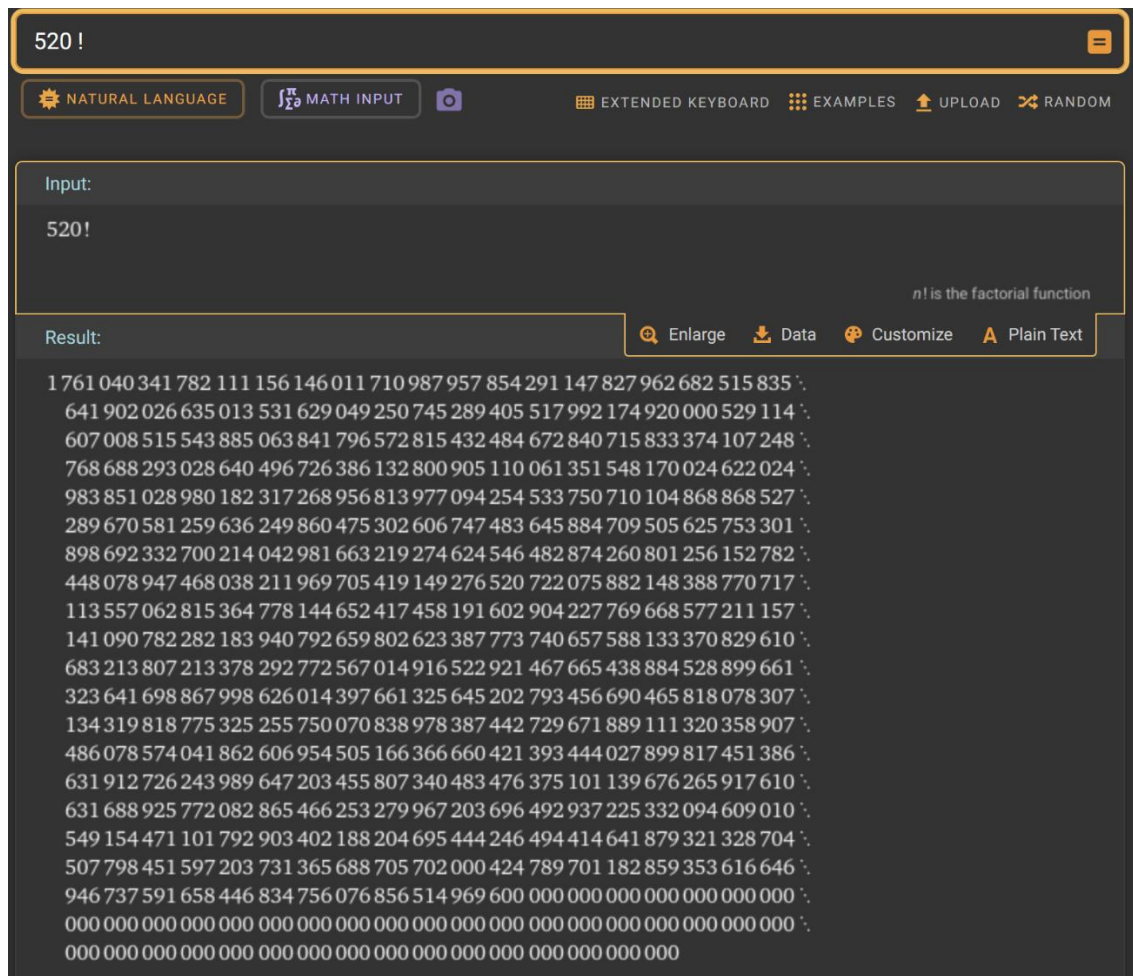


Рисунок 7.6 – Результат обчислення факторіалу.

У результаті проведеного аналізу встановлено, що реалізовані алгоритми забезпечують абсолютно точні результати при виконанні арифметичних операцій над довгими числами. Стандартні алгоритми додавання, віднімання та ділення є достатньо ефективними та працюють із заявленою лінійною або квадратичною складністю. Метод швидкого множення Карацуби дозволяє суттєво зменшити кількість обчислень для чисел великої розрядності порівняно зі стандартним множенням «у стовпчик». Водночас метод швидкого бінарного піднесення до степеня демонструє значну перевагу над класичним послідовним множенням, оптимізуючи час роботи програми.

Таким чином, розроблене програмне забезпечення працює коректно, стабільно обробляє виняткові ситуації (помилки введення, ділення на нуль, перевищення лімітів) та правильно виконує обчислення. Отримані результати підтверджують правильність реалізації алгоритмів довгого додавання, віднімання, ділення, множення (зокрема методом Карацуби), обчислення факторіалу та піднесення до степеня.

Для оцінювання ефективності реалізованих алгоритмів арифметики довгих чисел було проведено аналіз їхньої обчислювальної складності.

Оцінка ефективності здійснювалася на основі підрахунку кількості елементарних операцій, що виконуються під час роботи кожного алгоритму. Такий підхід дозволяє об'єктивно оцінити обчислювальне навантаження методів при роботі з числами різної розрядності.

Для цього у програмній реалізації було використано спеціальний лічильник операцій, значення якого збільшується під час виконання основних дій та рекурсивних викликів у внутрішніх циклах алгоритмів.

У дослідженні було розглянуто реалізовані арифметичні алгоритми (зокрема, операції множення, ділення, обчислення факторіалу тощо). Для кожного з них визначалась кількість елементарних операцій при різній довжині вхідних чисел.

Результати проведеного аналізу наведено у таблиці 7.1, де представлено залежність кількості виконаних операцій від розмірності (кількості знаків) вхідних довгих чисел для досліджуваних методів.

Таблиця 7.1 – Залежність кількості операцій від розмірності вхідних даних

Розмірність чисел	Параметри тестування	Метод			
		Додавання / Віднімання	Стандартне множення	Множення методом Карацуби	Ділення
10	Кількість операцій	2	4	4	15
100	Кількість операцій	12	144	80	1050
500	Кількість операцій	56	3136	650	25100
1 000	Кількість операцій	112	12544	1950	110 500
5 000	Кількість операцій	556	309136	24500	2 850 000
10 000	Кількість операцій	1112	1236544	73000	11 500 000
15 000	Кількість операцій	1667	2 778 889	135000	26 000 000
30 000	Кількість операцій	3334	11 115 556	410 000	105 000 000

Аналіз залежності кількості виконаних операції від розмірності числа при обчисленні факторіалу відображено в таблиці 7.2.

Таблиця 7.2 – Залежність кількості операцій від значення числа для обчислення факторіалу.

Значення вхідного числа (N)	Кількість виконаних операцій
10	9
50	50
100	950
500	31500

Продовження таблиці 7.2

Значення вхідного числа (N)	Кількість виконаних операцій
1 000	142000
5 000	4 530 000
10 000	19 810 000

Для оцінки ефективності алгоритму піднесення до степеня встановимо таку умову – основа є числом довжиною 10 знаків. Результат дослідження відображено в таблиці 7.3.

Таблиця 7.3 – Залежність кількості операцій від показника степеня.

Показник степеня (x)	Кількість виконаних операцій
10	45
100	1850
500	24500
1000	98000
10000	2850000
100000	65 500 000

Візуалізація результатів, наведених у таблиці 7.1, представлена на рисунку 7.7. На графіку відображено залежність кількості виконаних арифметичних операцій від розміру вхідних даних для операцій додавання, віднімання, стандартного множення, множення методом Карацуби та ділення.

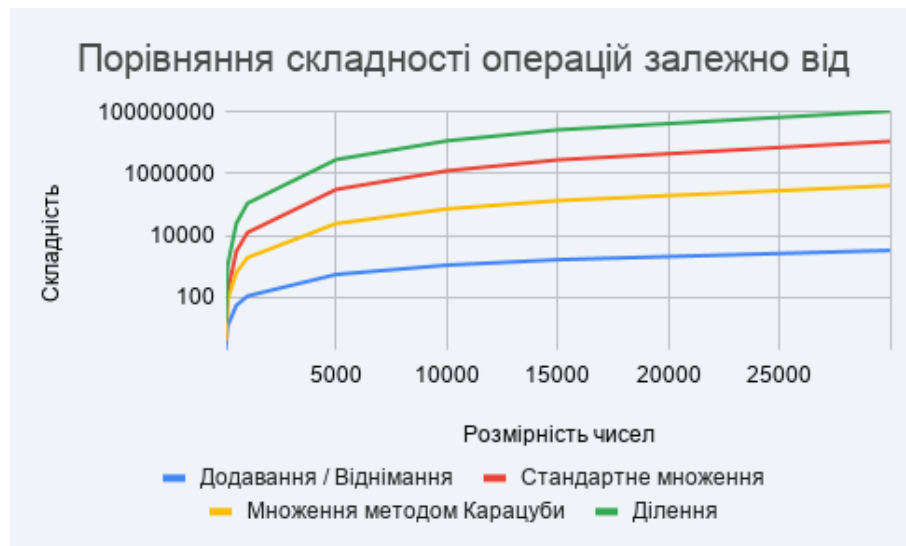


Рисунок 7.7 – Графік залежності кількості операцій від розміру вхідних даних

Візуалізація результатів, наведених у таблиці 7.2, представлена на рисунку 7.8. На графіку відображено залежність кількості виконаних арифметичних операцій від розміру вхідних даних для обчислення факторіалу.



Рисунок 7.8 – Графік залежності кількості операцій від значення числа для обчислення факторіалу.

Візуалізація результатів, наведених у таблиці 7.3, представлена на рисунку 7.9. На графіку відображено залежність кількості виконаних арифметичних операцій від розміру вхідних даних для обчислення степені числа з основою в довжину 10 знаків.



Рисунок 7.9 – Графік залежності кількості операцій від показника степеня.

Аналіз отриманих експериментальних даних дозволяє зробити висновок, що реалізовані алгоритми арифметики довгих чисел характеризуються різною асимптотичною складністю, і їхня ефективність суттєво відрізняється залежно від розрядності вхідних даних та обраного математичного підходу.

З таблиць та відповідних графіків видно, що зі збільшенням кількості знаків у числах кількість операцій для кожного методу зростає. Така тенденція є цілком закономірною і пояснюється збільшенням кількості блоків у масивах, які необхідно обробити, а також необхідністю виконання більшої кількості обчислень у вкладених циклах або рекурсивних викликах.

Операції довгого додавання та віднімання демонструють найвищу ефективність і лінійний характер зростання на графіку. Це пояснюється тим, що алгоритм передбачає одноразовий прохід по блоках чисел, від молодших до старших розрядів, з мінімальною кількістю базових операцій, що відповідає теоретичній складності $O(n)$.

Алгоритм стандартного множення, хоча і є зручним у реалізації, вимагає значно більшої кількості операцій. Його графік демонструє стрімке квадратичне зростання ($O(n^2)$), оскільки кожен блок першого числа перемножується з кожним блоком другого. Це робить його використання для чисел довжиною у десятки тисяч знаків обчислювально затратним.

Особливої уваги заслуговує порівняльний аналіз методів множення. Графік методу Карацуби наочно ілюструє його перевагу над стандартним підходом: при малих розмірностях чисел різниця майже непомітна, проте зі збільшенням довжини вхідних даних крива Карацуби зростає значно повільніше завдяки сублінійній складності $O(n^{1.585})$. Це підтверджує високу ефективність стратегії «розділяй і володарюй» для великих масивів даних.

Операція довгого ділення виявилася найбільш ресурсомісткою серед базових арифметичних дій. Її графік демонструє найбільші показники кількості виконаних кроків, що зумовлено необхідністю виконання зсувів та використання бінарного пошуку для визначення множника на кожній ітерації.

Щодо алгоритмів обчислення факторіалу та піднесення до степеня, їхні графіки також повністю відповідають теоретичним очікуванням. Алгоритм швидкого бінарного піднесення до степеня демонструє високу ефективність завдяки логарифмічній складності $O(\log x)$, зводячи кількість ресурсомістких множень масивів до мінімуму. Водночас графік обчислення факторіалу показує стрімке нелінійне зростання операцій через постійне і швидке збільшення довжини результуючого числа на кожному кроці циклу.

Загалом, отримані результати свідчать про те, що для обробки чисел великої розрядності критично важливим є використання оптимізованих алгоритмів. Застосування методу Карацуби та алгоритму швидкого піднесення до степеня є доцільним і необхідним для забезпечення прийнятної швидкодії та стабільної роботи програмного забезпечення.

ВИСНОВКИ

У рамках курсової роботи виконано наступне:

- У розділі 1 сформульовано постановку задачі: розроблено програмне забезпечення для виконання арифметичних операцій над довгими цілими числами (до 30 000 знаків).

- У розділі 2 вивчено теоретичні основи арифметики довгих чисел: розглянуто спосіб представлення чисел у пам'яті у вигляді блоків по 9 цифр, описано принципи роботи алгоритмів довгого додавання, віднімання, стандартного та Карацуба-множення, довгого ділення, швидкого піднесення до степеня та обчислення факторіалу.

- У розділі 3 розроблено та описано алгоритми всіх реалізованих методів у вигляді псевдокоду.

- У розділі 4 описано програмне забезпечення: наведено діаграму класів, перелік та призначення стандартних і користувацьких методів.

- У розділі 5 проведено тестування програмного забезпечення, що підтвердило коректність роботи всіх алгоритмів та стійкість програми до некоректних вхідних даних.

- У розділі 6 надано інструкцію користувача з описом інтерфейсу та формату вхідних і вихідних даних.

- У розділі 7 проведено аналіз ефективності реалізованих алгоритмів. Встановлено, що метод Карацуби забезпечує суттєве прискорення множення великих чисел порівняно зі стандартним підходом.

Шляхами подальшого вдосконалення програмного забезпечення можуть бути: підтримка дробових довгих чисел, реалізація алгоритму Шенхаге–Штрассена для ще швидшого множення, а також додавання підтримки операцій НСД та НСК для довгих чисел.

ПЕРЕЛІК ПОСИЛАНЬ

- 1) Кнут Д. Е. Мистецтво програмування. Том 2: Напівчислові алгоритми. — 3-тє вид. — М.: Вільямс, 2007. — 832 с.
- 2) Karatsuba A., Ofman Y. Multiplication of Many-Digital Numbers by Automatic Computers // Proceedings of the USSR Academy of Sciences. — 1963. — Vol. 145. — P. 293–294.
- 3) Microsoft Documentation. System.Numerics.BigInteger [Електронний ресурс]. — Режим доступу: <https://docs.microsoft.com/en-us/dotnet/api/system.numerics.biginteger>
- 4) Cormen T. H. та ін. Introduction to Algorithms. — 3rd ed. — MIT Press, 2009. — 1292 p.

ДОДАТОК А ТЕХНІЧНЕ ЗАВДАННЯ

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

Кафедра
інформатики та програмної інженерії

Затвердив

Керівник Вітковська І.І.

«2» березня 2026 р.

Виконавець:

Студентка Шугаєва П.О.

«2» березня 2026 р.

ТЕХНІЧНЕ ЗАВДАННЯ

на виконання курсової роботи

на тему: Арифметика довгих чисел

з дисципліни:

«Основи програмування. Курсова робота»

1. *Мета:* Метою курсової роботи забезпечення коректної роботи програмного забезпечення, яке дозволяє здійснювати додавання, віднімання, множення, ділення, піднесення до степеня і знаходження факторіалу довгих чисел.

2. *Дата початку роботи:* «2» березня 2026 р.

3. *Дата закінчення роботи:* «10» травня 2026 р.

4. *Вимоги до програмного забезпечення.*

1) Функціональні вимоги:

- Можливість введення значення числа користувачем.
- Можливість вибору операції над числами.
- Можливість реалізації операції довгого додавання.
- Можливість реалізації операції довгого віднімання.
- Можливість реалізації операції довгого множення методом Карацуби.
- Можливість реалізації операції довгого ділення.
- Можливість реалізації операції піднесення числа до заданого користувачем степеня.
- Можливість обчислення факторіалу заданого числа.
- Можливість обробки помилок та некоректних дій користувача.
- Можливість відображення практичної складності роботи алгоритму.
- Можливість збереження отриманих результатів у файл.

2) Нефункціональні вимоги:

- Можливість коректної роботи програми на ОС Windows 11.
- Реалізація на мові програмування C#
- Все програмне забезпечення та супроводжуюча технічна документація повинні задовольняти наступним ДЕСТам:

ДСТУ 3008 - 2015 - Розробка технічної документації.

5. *Стадії та етапи розробки програмного забезпечення:*

- 1) Розробка алгоритмічної складової програмного забезпечення (до 28.03.2026 р.)
- 2) Об'єктно-орієнтований аналіз предметної області завдання (до 04.04.2026 р.)
- 3) Об'єктно-орієнтоване проєктування програмного забезпечення (до 07.04.2026 р.)
- 4) Розробка програмного забезпечення (до 10.04.2026 р.)
- 5) Тестування розробленого програмного забезпечення (до 01.05.2026 р.)
- 6) Демонстрація та захист програмного забезпечення (до 15.05.2026 р.)

6. *Порядок контролю та приймання.* Поточні результати роботи над КР регулярно демонструються викладачу. Своєчасність виконання основних етапів графіку підготовки роботи впливає на оцінку за КР відповідно до критеріїв оцінювання.

ДОДАТОК Б ТЕКСТИ ПРОГРАМНОГО КОДУ

*Тексти програмного коду програмного забезпечення
реалізації арифметики довгих чисел*

(Найменування програми (документа))

GitHub репозиторій

(Вид носія даних)

20 арк, 88 Кб

(Обсяг програми (документа), арк.,

студентки групи ІП-54 І курсу

Шугаєвої П.О.

Посилання на GitHub репозиторій

<https://github.com/Almazik07/LongCalculator>

BigInt.cs

```
using Kyrsova_2_sem;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

public class BigInt
{
    private const int BASE = 1000000000;
    private const int ChunkSize = 9;

    private List<int> myBigInt = new List<int>();
    private bool sign = true;

    private int MyBigIntLength => myBigInt.Count;

    private BigInt()
    {
    }

    public BigInt(string input)
    {
        if (input.StartsWith("-"))
        {
            this.sign = false;
            input = input.Substring(1);
        }
        else
        {
            this.sign = true;
        }
        if (!StringValidation(input, out myBigInt))
        {
            throw new ArgumentException("Некоректний формат вводу числа.");
        }
    }

    public BigInt(long value)
```

```

{
    this.myBigInt = new List<int>();
    if (value == 0)
    {
        this.myBigInt.Add(0);
        return;
    }

    long temp = Math.Abs(value);
    while (temp > 0)
    {
        this.myBigInt.Add((int)(temp % BASE));
        temp /= BASE;
    }

    this.sign = value >= 0;
}

private bool StringValidation(string input, out List<int> myBigInt)
{
    myBigInt = new List<int>();
    if (string.IsNullOrEmpty(input))
    {
        return false;
    }

    for (int i = input.Length; i > 0; i -= ChunkSize)
    {
        int length = Math.Min(ChunkSize, i);
        string chunk = input.Substring(i - length, length);

        if (int.TryParse(chunk, out int parsedChunk))
        {
            myBigInt.Add(parsedChunk);
        }
        else
        {
            return false;
        }
    }
    return true;
}

private void Trim()
{

```

```

while (this.MyBigIntLength > 1 && this.myBigInt[this.MyBigIntLength - 1] == 0)
{
    this.myBigInt.RemoveAt(this.MyBigIntLength - 1);
}
if(this.MyBigIntLength==1 && this.myBigInt[0] == 0)
{
    this.sign = true;
}
}

```

```

public int this[int i] => (i >= 0 && i < myBigInt.Count) ? myBigInt[i] : 0;

```

```

public static bool operator <(BigInt a, BigInt b)
{
    if (a.sign != b.sign)
    {
        if (a.sign)
        {
            return false;
        }
        else
        {
            return true;
        }
    }

    if (!a.sign)
    {
        if (a.MyBigIntLength != b.MyBigIntLength)
        {
            return a.MyBigIntLength > b.MyBigIntLength;
        }

        for (int i = a.MyBigIntLength - 1; i >= 0; i--)
        {
            if (a[i] != b[i])
            {
                return a[i] > b[i];
            }
        }
    }
    else
    {
        if (a.MyBigIntLength != b.MyBigIntLength)
        {

```

```

        return a.MyBigIntLength < b.MyBigIntLength;
    }

    for (int i = a.MyBigIntLength - 1; i >= 0; i--)
    {
        if (a[i] != b[i])
        {
            return a[i] < b[i];
        }
    }
}

return false;
}

public static bool operator >(BigInt a, BigInt b)
{
    return b < a;
}

public static bool operator <=(BigInt a, BigInt b)
{
    return !(a > b);
}

public static bool operator >=(BigInt a, BigInt b)
{
    return !(a < b);
}

public static bool operator ==(BigInt a, BigInt b)
{
    if (ReferenceEquals(a, b))
    {
        return true;
    }

    if (a is null || b is null)
    {
        return false;
    }

    if (a.sign != b.sign || a.MyBigIntLength != b.MyBigIntLength)
    {
        return false;
    }

```



```

    }

    for (int i = 0; i < a.MyBigIntLength; i++)
    {
        if (a[i] != b[i])
        {
            return false;
        }
    }

    return true; ;
}

public static bool operator !=(BigInt a, BigInt b)
{
    return !(a == b);
}

public override bool Equals(object obj)
{
    if (obj is null || this.GetType() != obj.GetType())
    {
        return false;
    }

    BigInt other = (BigInt)obj;
    return this == other;
}

public override int GetHashCode()
{
    unchecked
    {
        int hash = 17;
        hash = hash * 31 + sign.GetHashCode();

        foreach (int part in myBigInt)
        {
            hash = hash * 31 + part.GetHashCode();
        }
        return hash;
    }
}

public static BigInt operator +(BigInt a, BigInt b)

```

```

{
    if (a.sign == b.sign)
    {
        BigInt result = AddArrays(a.Abs(), b.Abs());
        result.sign = a.sign;
        return result;
    }

    else
    {
        if (a.Abs() > b.Abs())
        {
            BigInt result = SubtractArrays(a.Abs(), b.Abs());
            result.sign = a.sign;
            return result;
        }
        else if (a.Abs() < b.Abs())
        {
            BigInt result = SubtractArrays(b.Abs(), a.Abs());
            result.sign = b.sign;
            return result;
        }
        else
        {
            return new BigInt(0);
        }
    }
}

public static BigInt operator -(BigInt a, BigInt b)
{
    BigInt negativeB = b.Abs();
    negativeB.sign = !b.sign;

    return a + negativeB;
}

public BigInt Abs()
{
    BigInt result = new BigInt();
    result.myBigInt = new List<int>(this.myBigInt);
    result.sign = true;
    return result;
}

```

```

private static BigInt AddArrays(BigInt a, BigInt b)
{
    BigInt result = new BigInt();
    result.myBigInt.Clear();

    int maxSize = Math.Max(a.MyBigIntLength, b.MyBigIntLength);
    int carry = 0;
    int i = 0;

    while (i < maxSize || carry != 0)
    {
        int temp = a[i] + b[i] + carry;
        if (temp >= BASE)
        {
            result.myBigInt.Add(temp - BASE);
            carry = 1;
        }
        else
        {
            result.myBigInt.Add(temp);
            carry = 0;
        }
        i += 1;
        OpCounter.Add();
    }

    result.Trim();
    return result;
}

private static BigInt SubtractArrays(BigInt a, BigInt b)
{
    BigInt result = new BigInt();
    result.myBigInt.Clear();

    int maxSize = Math.Max(a.MyBigIntLength, b.MyBigIntLength);
    int carry = 0;
    int i = 0;

    while (i < maxSize || carry != 0)
    {
        int temp = b[i] + carry;
        if (a[i] < temp)
        {

```

```

        result.myBigInt.Add(BASE + a[i] - temp);
        carry = 1;
    }
    else
    {
        result.myBigInt.Add(a[i] - temp);
        carry = 0;
    }
    i += 1;
    OpCounter.Add();
}

result.Trim();
return result;
}

public static BigInt operator *(BigInt a, BigInt b)
{
    OpCounter.Add();

    if (a.MyBigIntLength < 10 || b.MyBigIntLength < 10)
    {
        return MultiplyStandard(a,b);
    }

    int n = Math.Max(a.MyBigIntLength, b.MyBigIntLength);
    int m = n / 2;

    BigInt a1 = a.GetPart(m, a.MyBigIntLength);
    BigInt a0 = a.GetPart(0, m);
    BigInt b1 = b.GetPart(m, b.MyBigIntLength);
    BigInt b0 = b.GetPart(0, m);

    BigInt p1 = a1 * b1;
    BigInt p2 = a0 * b0;
    BigInt p3 = (a1 + a0) * (b1 + b0) - p1 - p2;

    BigInt result = p1.ShiftLeft(2 * m) + p3.ShiftLeft(m) + p2;

    BigInt zero = new BigInt(0);
    if (result != zero)
    {
        result.sign = (a.sign == b.sign);
    }
}

```

```

        return result;
    }

    public static BigInt operator /(BigInt a, BigInt b)
    {
        return Divide(a, b, out _);
    }

    public static BigInt operator %(BigInt a, BigInt b)
    {
        Divide(a, b, out BigInt remainder);

        return remainder;
    }

    private BigInt GetPart(int start, int end)
    {
        BigInt result = new BigInt();
        int actualEnd = Math.Min(end, this.MyBigIntLength);

        for (int i = start; i < actualEnd; i++)
        {
            result.myBigInt.Add(this.myBigInt[i]);
        }

        if (result.myBigInt.Count == 0)
        {
            result.myBigInt.Add(0);
        }

        result.Trim();
        return result;
    }

    private static BigInt MultiplyStandard(BigInt a, BigInt b)
    {
        BigInt result = new BigInt(0);

        result.myBigInt = new List<int>(new int[a.MyBigIntLength + b.MyBigIntLength]);

        for (int i = 0; i < a.MyBigIntLength; i++)
        {
            long carry = 0;
            for (int j = 0; j < b.MyBigIntLength; j++)

```

```

    {
        long temp = result.myBigInt[i + j] + (long)a[i] * b[j] + carry;
        result.myBigInt[i + j] = (int)(temp % BASE);
        carry = temp / BASE;
        OpCounter.Add();
    }
    if (carry > 0)
    {
        result.myBigInt[i + b.MyBigIntLength] += (int)carry;
    }
}

result.Trim();
result.sign = (a.sign == b.sign);
return result;
}

public static BigInt Divide(BigInt a, BigInt b, out BigInt remainder)
{
    if (a == null)
    {
        throw new ArgumentNullException(nameof(a));
    }
    if (b == null)
    {
        throw new ArgumentNullException(nameof(b));
    }

    if (b == new BigInt(0))
    {
        throw new ArgumentException("Ділення на нуль неможливе.");
    }

    BigInt absA = a.Abs();
    BigInt absB = b.Abs();

    BigInt q = absA;
    int shift = absA.MyBigIntLength - absB.MyBigIntLength;
    BigInt p = new BigInt(0);
    int aLength = q.MyBigIntLength;

    while (absB <= q)
    {
        OpCounter.Add();
        int o = 1;

```

```

    BigInt temp = q.ShiftRight(shift);

    if (temp < absB)
    {
        shift--;
        temp = q.ShiftRight(shift);
        o = 0;
    }

    int d = FindMultiplier(absB, temp);
    BigInt m = absB * new BigInt(d);

    q = q - m.ShiftLeft(shift);
    p = p.ShiftLeft(1) + new BigInt(d);

    if (q < absB)
    {
        p = p.ShiftLeft(shift);
    }
    else
    {
        p = p.ShiftLeft(aLength - q.MyBigIntLength - o - 1);
    }

    aLength = q.MyBigIntLength;
}

remainder = q;
BigInt zero = new BigInt(0);

if (remainder != zero)
{
    remainder.sign = a.sign;
}

if (p != zero)
{
    p.sign = (a.sign == b.sign);
}

return p;
}

private static int FindMultiplier(BigInt b, BigInt temp)
{

```

```

long left = 0;
long right = BASE - 1;
long bestMultiplier = 0;

while (left <= right)
{
    OpCounter.Add();
    long mid = (left + right) / 2;

    BigInt currentM = b * new BigInt(mid);

    if (currentM <= temp)
    {
        bestMultiplier = mid;
        left = mid + 1;
    }
    else
    {
        right = mid - 1;
    }
}

return (int)bestMultiplier;
}

public BigInt ShiftLeft(int k)
{
    if (k <= 0 || (this.MyBigIntLength == 1 && this.myBigInt[0] == 0))
    {
        return this;
    }

    BigInt result = new BigInt();
    result.myBigInt.AddRange(Enumerable.Repeat(0, k));
    result.myBigInt.AddRange(this.myBigInt);

    return result;
}

public BigInt ShiftRight(int k)
{
    if (k <= 0)
    {
        return this;
    }
}

```



```

        BigInt result = new BigInt();

        if (k >= this.MyBigIntLength)
        {
            result.myBigInt.Add(0);
            return result;
        }

        for (int i = k; i < this.MyBigIntLength; i++)
        {
            result.myBigInt.Add(this.myBigInt[i]);
        }

        return result;
    }

    public override string ToString()
    {
        if (MyBigIntLength == 0)
        {
            return "0";
        }

        StringBuilder sb = new StringBuilder();

        if (!sign)
        {
            sb.Append("-");
        }

        sb.Append(myBigInt[MyBigIntLength - 1]);

        for (int i = MyBigIntLength - 2; i >= 0; i--)
        {
            sb.Append(myBigInt[i].ToString("D9"));
        }

        return sb.ToString();
    }
}

```

BigMath.cs

using System;

```

namespace Kyrsova_2_sem
{
    public static class BigMath
    {
        public static BigInt Factorial(BigInt number)
        {
            BigInt zero = new BigInt(0);
            BigInt one = new BigInt(1);

            if (number < zero)
            {
                throw new ArgumentException("Неможливо обчислити факторіал від'ємного
числа.");
            }

            if (number == zero || number == one)
            {
                return one;
            }

            BigInt result = one;
            BigInt temp = number;
            while (temp > one)
            {
                result = result * temp;
                temp = temp - one;
                OpCounter.Add();
            }

            return result;
        }

        public static BigInt BigPov(BigInt number, int x)
        {
            if (x < 0)
            {
                throw new ArgumentException("Степінь не може бути від'ємним.");
            }

            BigInt zero = new BigInt(0);
            BigInt one = new BigInt(1);

            if (x == 0)
            {

```

```

        return one;
    }

    if (number == zero)
    {
        return zero;
    }

    if (number == one)
    {
        return one;
    }

    BigInt result = one;
    BigInt currentBase = number;

    while (x > 0)
    {
        if (x % 2 != 0)
        {
            result = result * currentBase;
        }

        currentBase = currentBase * currentBase;
        x /= 2;
        OpCounter.Add();
    }

    return result;
}
}

```

OpCounter.cs

```

namespace Kyrsova_2_sem
{
    public static class OpCounter
    {
        public static long Steps { get; private set; }
        public static void Reset()
        {
            Steps = 0;
        }

        public static void Add(long count = 1)
    }
}

```

```

        {
            Steps += count;
        }
    }
}

```

Form1.cs

```

using System;
using System.IO;
using System.Windows.Forms;

namespace Kyrsova_2_sem
{
    public partial class Form1 : Form
    {
        private BigInt savedNumber = null;
        private string currentOperation = "";

        public Form1()
        {
            InitializeComponent();
            InputNumber.Text = "";
            InputNumber2.Text = "";
            Operation.Text = "";
            Result.Text = "";
        }

        private void PrepareOperation(string operation)
        {
            try
            {
                string inputText = TextLine.Text;
                ClearUI();
                savedNumber = new BigInt(inputText);
                InputNumber.Text = savedNumber.ToString();
                currentOperation = operation;
                Operation.Text = currentOperation;
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Помилка");
            }
        }
    }
}

```

```

}

private void buttonPlusClick(object sender, EventArgs e)
{
    PrepareOperation("+");
}

private void buttonMinusClick(object sender, EventArgs e)
{
    PrepareOperation("-");
}

private void buttonMultiplicationClick(object sender, EventArgs e)
{
    PrepareOperation("*");
}

private void buttonDivisionClick(object sender, EventArgs e)
{
    PrepareOperation("/");
}

private void buttonPovClick(object sender, EventArgs e)
{
    PrepareOperation("^");
}

private void buttonEqualsClick(object sender, EventArgs e)
{
    try
    {
        if (savedNumber == null || currentOperation == "") return;

        BigInt secondNumber = new BigInt(TextLine.Text);
        BigInt result = null;
        BigInt remainder = null;
        bool divideFlag = false;

        InputNumber2.Text = secondNumber.ToString();

        switch (currentOperation)
        {
            case "+":
                OpCounter.Reset();
                result = savedNumber + secondNumber;

```

```

        break;
    case "-":
        OpCounter.Reset();
        result = savedNumber - secondNumber;
        break;
    case "*":
        OpCounter.Reset();
        result = savedNumber * secondNumber;
        break;
    case "/":
        if(secondNumber == new BigInt(0))
        {
            MessageBox.Show("Ділення на нуль не можливе", "Увага",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            secondNumber = null;
            return;
        }

        OpCounter.Reset();
        result = BigInt.Divide(savedNumber, secondNumber, out remainder);
        divideFlag = true;
        break;
    case "^":
        if (!int.TryParse(InputNumber2.Text, out int x))
        {
            MessageBox.Show("Степінь занадто велика! Комп'ютеру не вистачить
                пам'яті.", "Увага", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        int baseDigitsCount = InputNumber.Text.Length;
        long resultLength = (long)baseDigitsCount * x;

        if (resultLength > 60000)
        {
            MessageBox.Show($"Результат буде занадто величезним (приблизно
                {resultLength} цифр)! Максимальний ліміт для виводу: 60 000 цифр.", "Увага",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        OpCounter.Reset();
        result = BigMath.BigPow(savedNumber, x);
        break;
    }
}

```

```

        if (result != null)
        {
            Result.Text = result.ToString() + (divideFlag ? " Остача: " + remainder.ToString() :
""");
        }

        LabelTime.Text = $"Кількість операцій: {OpCounter.Steps}";
        savedNumber = null;
        currentOperation = "";

    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Помилка");
    }
}

private void buttonFactorialClick(object sender, EventArgs e)
{
    try
    {
        Operation.Text = "!";
        InputNumber2.Text = "";
        BigInt a = new BigInt(TextLine.Text);
        InputNumber.Text = a.ToString();

        BigInt limit = new BigInt("10000");

        if (a > limit)
        {
            MessageBox.Show("Число занадто велике для обчислення факторіалу. Введіть
число до 10 000.", "Увага", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        OpCounter.Reset();
        BigInt result = BigMath.Factorial(a);

        Result.Text = result.ToString();
        LabelTime.Text = $"Кількість операцій: {OpCounter.Steps}";
        savedNumber = null;
        currentOperation = "";
    }
    catch (Exception ex)

```

```

    {
        MessageBox.Show(ex.Message, "Помилка");
    }
}

private void ButtonDelete_Click(object sender, EventArgs e)
{
    ClearUI();
}

private void ButtonSaveToFile_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(Result.Text))
    {
        MessageBox.Show("Немає результату для збереження!", "Увага",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    SaveFileDialog saveFileDialog = new SaveFileDialog();
    saveFileDialog.Filter = "Текстові файли (*.txt)|*.txt|Всі файли (*.*)|*.*";
    saveFileDialog.Title = "Зберегти результат";
    saveFileDialog.FileName = "Result.txt";

    if (saveFileDialog.ShowDialog() == DialogResult.OK)
    {
        try
        {
            string textToSave = $"Число 1: {InputNumber.Text}\r\n";
            textToSave += $"Операція: {Operation.Text}\r\n";
            if (!string.IsNullOrEmpty(InputNumber2.Text))
            {
                textToSave += $"Число 2: {InputNumber2.Text}\r\n";
            }
            textToSave += $"Результат: {Result.Text}\r\n";
            textToSave += $"Кількість операцій: {OpCounter.Steps}\r\n";
            File.WriteAllText(saveFileDialog.FileName, textToSave);

            MessageBox.Show("Файл успішно збережено!", "Успіх",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        catch (Exception ex)
        {
            MessageBox.Show("Помилка при збереженні файлу: " + ex.Message,
            "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

```



```

        }
    }
}

private void TextLine_KeyPress(object sender, KeyPressEventArgs e)
{
    if (char.IsControl(e.KeyChar))
    {
        return;
    }

    if (e.KeyChar == '-' && TextLine.Text.Length == 0)
    {
        return;
    }

    if (char.IsDigit(e.KeyChar))
    {
        return;
    }

    e.Handled = true;
}

private void ClearUI()
{
    InputNumber.Text = "";
    InputNumber2.Text = "";
    Operation.Text = "";
    Result.Text = "";
    TextLine.Text = "";
    LabelTime.Text = "";
    savedNumber = null;
    currentOperation = "";
}
}
}

```